

Multi-Agent Large Language Models for Connectivity Restoration in Damaged Graphs*

Ruilin Wang^{1,3}, Xiaolin Ai^{1,2†}, and Yang Xu⁴

¹ National Key Laboratory of Cognition and Decision Intelligence for Complex Systems, Institute of Automation, Chinese Academy of Sciences

² National Key Laboratory of Information Systems Engineering

³ University of Chinese Academy of Sciences

⁴ School of Automation Science and Electrical Engineering, Beihang University
wangruilin231@mailsucas.ac.cn

Abstract. Restoring connectivity in graphs with damaged nodes or edges is critical for maintaining the functionality of real-world networked systems. Rule-based methods ensure structural validity but lack flexibility in heterogeneous topologies, while learning-based approaches often require task-specific training and may generalize poorly under shifted graph distributions. In this work, we study intent-aware graph restoration as a constrained strategy-selection and execution problem over a small, explicit library of repair strategies. We instantiate a collaborative multi-agent framework based on large language models, using the collaborative agent repair engine (CARE) architecture as the orchestration backbone and adapting it to graph repair through graph-specific semantic grounding, structural constraint extraction, and executable repair generation. Specialized agents, including a Terminology Interpreter and a Graph Expert, communicate through a shared Comment Pool under a Conductor, and a Reducer selects a feasible repair strategy from minimum-spanning-tree, bridge, and latency-oriented templates. Experiments on synthetic and real-world damaged networks show that the framework provides controllable natural-language intent alignment and competitive zero-shot repair behavior, especially on heterogeneous random graphs, while reinforcement learning remains strong on in-distribution geometric graphs. Ablation results further indicate that semantic grounding and structural validation play complementary roles in reducing strategy-selection errors and redundant repairs.

Keywords: Graph Repair, Large Language Models, Multi-Agent Systems

*This work was supported by the National Natural Science Foundation of China under Grant No. 62503472, the Young Scientists Foundation of CSAA (Guidance Navigation and Control, GNC) under Grant CSAA-YSF2025-GNC-08, and the Open Fund of National Key Laboratory of Information Systems Engineering (No. 6142101240203).

[†]Corresponding author.

1 Introduction

Graph repair is a fundamental challenge in network science, centered on restoring damaged topological structures resulting from data loss, erroneous connections, or operational noise. In critical systems such as communication infrastructures, transportation networks, and knowledge graphs, structural inconsistencies directly compromise reliability. Effective repair therefore requires strategic modifications to nodes or edges that satisfy structural constraints while optimizing connectivity, efficiency, and cost.

Traditional methodologies primarily rely on rule-based or constraint-driven frameworks. For instance, Cheng et al. [1] utilized a dichotomy of strict and flexible rules to balance mandatory constraints with cost minimization. Lauer et al. [3] leveraged graph transformation systems to automate repairs while mitigating unintended side effects. Furthermore, local repair strategies and incremental sampling have been developed to optimize navigability in dynamic environments [7]. Despite their precision, these approaches often depend on hand-crafted rules and rigid procedures, limiting their adaptability to heterogeneous graph structures or evolving objectives.

Learning-based methods offer a more dynamic alternative, and recent advances in large language models suggest promise for reasoning-intensive structural optimization [6]. Unlike task-specific training pipelines, LLMs can interpret natural-language descriptions and synthesize decision strategies from context. A notable development is the collaborative agent repair engine, framework [5], which uses specialized-agent collaboration to decompose complex operations-research problems.

Building on these insights, we adapt the CARE orchestration principle to damaged-graph connectivity restoration. Our goal is not to claim a new general-purpose multi-agent architecture; rather, we examine how a CARE-style framework can be instantiated for graph repair by separating semantic intent interpretation from graph-structural constraint reasoning. This distinction is important because the present system operates over a finite strategy library, rather than performing fully open-ended graph synthesis. In this paper, the library contains three repair templates: minimum-spanning-tree, bridge-based connectivity restoration, and latency-oriented shortcut insertion. The technical focus is therefore intent-aware, constraint-aware strategy selection and executable repair generation under a controlled graph-repair setting.

Our primary contributions are as follows. (1) We formulate graph restoration as a library-based strategy-selection problem that connects natural-language intent, graph constraints, and executable repair actions. (2) We provide a graph-specific CARE instantiation with a Terminology Interpreter for semantic grounding, a Graph Expert for structural validation, and a Reducer that selects among explicit repair strategies using the shared Comment Pool. (3) We present experiments on synthetic and real-world damaged graphs, including ablations, score definitions, prompt protocol details, and an analysis of when CARE is effective relative to heuristic and reinforcement-learning baselines.

2 Related Work

Graph restoration has been extensively studied across network science and database optimization, spanning three primary paradigms: rule-based enforcement, learning-based optimization, and emerging LLM-based reasoning.

Rule-Based Graph Repair: Rule-based methods prioritize structural consistency by defining logical constraints and searching for minimum-cost transformations. Cheng et al. [1] implemented algorithms that guarantee correctness across strict and flexible rule sets. Lauer et al. [3] refined this through graph transformation rules to ensure repairs preserve underlying model semantics. Expanding on traditional methods, Zhao and Chen [7] proposed local connectivity repair and incremental sampling to optimize graph navigability in dynamic settings. While effective for enforcing known constraints, these systems lack flexibility and usually require manual re-engineering to handle novel structural patterns or environments.

Reinforcement Learning in Graph Optimization: To address adaptability, reinforcement learning has been applied to guide local searches and select repair operations dynamically. Wang et al. [4], for instance, integrated an RL-based procedure to solve the minimum weight independent dominating set problem, demonstrating how agents can learn transition policies through experience. However, RL-based systems often incur substantial training overhead and can be sensitive to the graph distribution used during training.

LLMs for Operations Research and CARE: LLMs have introduced a zero-shot alternative for solving optimization problems without task-specific retraining. The CARE or Chain-of-Experts framework [5] represents a significant shift, using role-based reasoning to transform unstructured problem descriptions into formal models. Our work uses this framework as an orchestration backbone rather than proposing a fundamentally different agent architecture. The main distinction lies in the graph-repair instantiation: real graph data are exposed to a Graph Expert, terminology-level intent signals are constrained by a Terminology Interpreter, and the Reducer is restricted to a transparent strategy library. This design aims to reduce structural hallucination and make the scope of the decision process auditable.

3 Method

Our framework adapts the multi-agent orchestration principle of CARE [5] to graph-repair tasks, emphasizing the separation between semantic grounding and structural validation.

3.1 Problem Formulation and Scope

Given a damaged graph $G = (V, E)$ with corrupted nodes and edges, the goal is to construct a repaired graph $\hat{G} = (\hat{V}, \hat{E})$ that satisfies structural constraints

while optimizing a target objective. We explicitly constrain the current system to a small strategy library

$$\mathcal{S}_{base} = \{S_{mst}, S_{bridge}, S_{latency}\}, \quad (1)$$

where S_{mst} emphasizes low total connection cost through a component-level MST, S_{bridge} greedily connects disconnected components with short bridge edges, and $S_{latency}$ first restores connectivity and then adds a limited number of short-cuts to reduce path length. This scope makes the task a constrained strategy-selection problem rather than an open-ended graph-repair problem in which the system invents arbitrary new repair operators.

The Reducer selects a strategy according to

$$S^* = \arg \max_{S \in \mathcal{S}_{base}} \mathcal{F}_{LLM}(S \mid \mathcal{C}_{pool}, H), \quad (2)$$

where $\mathcal{C}_{pool}$ represents the aggregated insights within the shared Comment Pool, encompassing both semantic interpretations from the Terminology Interpreter and structural constraints extracted by the Graph Expert. H denotes optional human preferences or overrides. In practice, $\mathcal{F}_{LLM}$ is implemented as a prompt-based comparison using DeepSeek-V3. The prompt asks the model to evaluate candidate strategies based on (i) adherence to the synthesized constraints and semantic context in $\mathcal{C}_{pool}$, (ii) alignment with human preferences, and (iii) expected structural improvement. The experimental Score used for reporting performance is defined separately in Section 4.2; it is not used during strategy selection.

3.2 Modular Multi-Agent Framework for Graph Restoration

We design a multi-agent framework structured as CARE, which decomposes reasoning into modular components to handle the dual complexity of semantic interpretation and structural validation. The components are as follows.

Input Layer: The system first identifies the restoration goals for a damaged topological structure. A Knowledge Expert provides executable algorithm templates and natural-language descriptions for the three predefined strategies in $\mathcal{S}_{base}$.

Analysis Layer & Comment Pool: The Conductor dispatches tasks to specialized agents. The Terminology Interpreter maps descriptions and human preferences to domain-level optimization signals, while the Graph Expert extracts graph size, edge structure, forbidden edges, and must-connect edges from the structured input. Their outputs are deposited in a shared Comment Pool, ensuring that distinct analytical perspectives are retained for final synthesis by the Reducer.

Summary Layer: The Reducer aggregates the inputs from the Comment Pool alongside human preferences. It resolves conflicts among cost, connectivity, and path-efficiency signals, selects S^* from $\mathcal{S}_{base}$, and generates executable code that applies the selected strategy to the damaged graph.

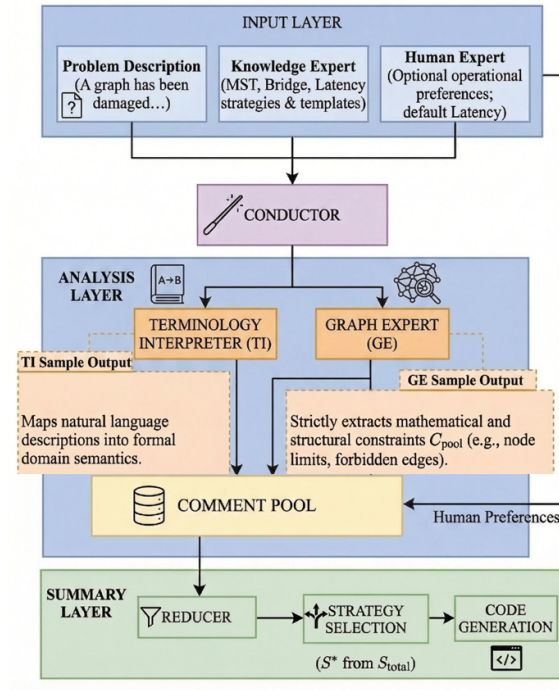


Fig. 1. Overview of the collaborative agent repair engine framework. The system consists of an input module, a shared Comment Pool for intermediate reasoning, and a processing module that performs constrained strategy selection and executable repair generation.

3.3 Algorithm

The process begins with the Conductor parsing the input description D and initializing the knowledge base K . To ensure structural integrity, the reasoning is bifurcated: the Terminology Interpreter clarifies semantic intent while the Graph Expert extracts hard structural constraints. These parallel insights are aggregated into a shared Comment Pool C_{pool} .

As formalized in Algorithm 1, the Reducer evaluates the collected context against human preferences H and default repair preferences across the predefined strategy set before the final selection phase. Finally, the system generates and executes code C_{exec} to produce the repaired graph $\hat{G}$.

Algorithm 1 Streamlined CARE Graph Repair Pipeline**Require:** Description D , Graph G , Knowledge K providing $\mathcal{S}_{base}$, Human Preference H **Ensure:** Executable Code C_{exec} , Repaired Graph $\hat{G}$

- 1: $\mathcal{C}_{pool} \leftarrow \text{Conductor}(D, K, H)$ ▷ Initialize context and parse requirements
- 2: $\mathcal{C}_{pool} \leftarrow \mathcal{C}_{pool} \cup \{\text{TI}(D), \text{GE}(G)\}$ ▷ Semantic and structural analysis
- 3: $H' \leftarrow H$ if $H \neq \emptyset$, otherwise default preferences: fewer added edges, no forbidden edges, and simple repairs
- 4: $S^* \leftarrow \arg \max_{S \in \mathcal{S}_{base}} \mathcal{F}_{LLM}(S \mid \mathcal{C}_{pool}, H')$
- 5: $C_{exec} \leftarrow \text{GenCode}(S^*, \mathcal{C}_{pool})$ ▷ Generate implementation for S^*
- 6: $\hat{G} \leftarrow \text{Execute}(C_{exec}, G)$
- 7: **return** $C_{exec}, \hat{G}$

4 Experiments

4.1 Experimental Setup

Prompt templates are kept consistent across all LLM-based stages to ensure a fair comparison. We evaluate the proposed CARE framework powered by DeepSeek-V3 on both synthetic and real-world graph-repair tasks. The Reducer prompt explicitly lists the three available strategies, includes human preferences and hard constraints, appends expert comments from the Comment Pool, and requires a valid JSON output containing the selected strategy and complete executable Python code. The Terminology Interpreter is instructed to map terms and preferences to optimization signals without directly choosing a strategy, while the Graph Expert is instructed to use only the provided nodes, edges, positions, forbidden edges, and must-connect edges.

The RL Strategy baseline is provided with the same graph constraints and is evaluated by the same reward function used for reporting Score. The comparison should therefore be interpreted as a deployment comparison under a shared evaluation protocol rather than an attempt to equalize training cost or supervision: RL receives task-specific training, whereas CARE performs zero-shot strategy selection from the library. All experiments use fixed random seeds and identical damaged instances across methods to make the reported comparisons reproducible. The results should thus be read as controlled relative behavior on representative settings, while broader multi-seed stability remains an important direction for future evaluation.

Synthetic Graphs: We generate random graphs consisting of $N \in \{50, 100\}$ nodes uniformly distributed within a unit square, where edge weights are defined as Euclidean distances between node pairs. Two graph models are considered: (i) Erdős-Rényi (ER) graphs, in which each edge is included independently with probability $p = 0.15$, and (ii) random geometric graphs (RGGs), where edges are formed between nodes whose Euclidean distance is within a connection radius of $R = 0.15$. To simulate structural damage, we introduce two corruption scenarios: mild damage and severe damage. This notation is reflected in the tables as “2 nodes + 5 edges” and “5 nodes + 10 edges” to avoid confusion with graph size.

Constraints: Repair must satisfy forbidden-edge constraints and must-connect constraints. If strict feasibility is impossible, the system falls back to minimal constraint violation while still prioritizing global connectivity.

4.2 Baselines and Metrics

We evaluate CARE against two representative baselines: Latency [2], a fixed heuristic adapted from post-disaster minimum-latency objectives and designed to reduce path length through shortcut edge insertion, and RL Strategy, a reinforcement-learning approach trained until convergence for approximately 200 epochs.

The evaluation encompasses five primary metrics. The GCC ratio measures global network connectivity, while Redundancy quantifies repair cost through normalized edge addition or, in the ablation tables, the number of added edges. Path efficiency is assessed via Average Shortest Path Length (ASPL) and Weighted ASPL (WAPL). Finally, an aggregate Score represents the overall utility of the repair:

$$\text{Score} = 2\Delta\text{GCC} + 0.5 \tanh(\Delta\text{ASPL}) + 0.5 \tanh(\Delta\text{WAPL}) - 0.2 \text{cost}_{\text{norm}} + P_{\text{fail}}, \quad (3)$$

where $\Delta\text{GCC} = \text{GCC}_{\text{after}} - \text{GCC}_{\text{before}}$, $\Delta\text{ASPL} = \text{ASPL}_{\text{before}} - \text{ASPL}_{\text{after}}$, $\Delta\text{WAPL} = \text{WAPL}_{\text{before}} - \text{WAPL}_{\text{after}}$, $\text{cost}_{\text{norm}}$ is the total Euclidean length of added edges normalized by the graph bounding-box diagonal, and $P_{\text{fail}} = -1$ if the repaired graph remains disconnected and 0 otherwise.

4.3 Intent-Driven Repair on a Real-World Topology

To evaluate the practical decision-making capability of CARE, we conduct a case study on a real-world communication network with 62 nodes under severe structural damage, created by removing 5 nodes and 10 edges. We provide CARE with three distinct human-preference prompts: (i) “Minimize total construction cost”, (ii) “Ensure robust connectivity”, and (iii) “Maximize communication efficiency”. The CARE framework maps these prompts to the intended strategies in the current library.

Quantitative results on the real-world topology are summarized in Table 1. We primarily compare against the learning-based RL baseline here to highlight the difference between zero-shot library-based reasoning and data-driven learning in a real-world setting; the Latency baseline is omitted from this table for space and because its behavior is already reported in the synthetic evaluation. The RL baseline shows a weaker Score on this representative instance. In contrast, the strategy auto-selected by CARE maintains stable structural metrics without retraining. As shown in Table 2, CARE achieves a 1.00 success rate in mapping the three natural-language prompts to the expected repair strategies.

Table 1. Quantitative Results on the Real-World Graph ($N = 62$)

Method	Score $\uparrow$	Redundancy $\downarrow$	ASPL $\downarrow$	WAPL $\downarrow$
CARE (Auto-selected)	0.3447	1.304	5.344	1342.66
RL Strategy	-0.5752	1.268	5.346	1343.78

Table 2. Strategy Selection Consistency and Human Control

Prompt	Selected Strategy	Expected Success Rate	
MST Prompt	MST	MST	1.00
Bridge Prompt	Bridge	Bridge	1.00
Latency Prompt	Latency	Latency	1.00

4.4 Reliability and Generalization

We compare CARE with the RL Strategy under different graph scales and topologies.

RL In-Distribution Performance: RL performs best on structured RGG graphs. At $N = 50$, it achieves the highest Scores, outperforming both Latency and CARE and showing strong adaptation to fixed geometric distributions. However, when $N = 100$, its advantage becomes smaller.

CARE Cross-Topology Robustness: CARE shows stable zero-shot behavior across heterogeneous topologies. It consistently selects the Bridge strategy because the Graph Expert prioritizes disconnected components and avoids unnecessary edge additions. This is effective for ER graphs, where a few bridge edges are often sufficient to restore connectivity. In RGG graphs, spatial locality causes longer detours, making shortcut-based policies such as RL and Latency more effective. This indicates that CARE is stronger for topology transfer, while RL is preferable when test graphs match its training distribution.

Table 3. Performance Comparison Across Scales and Damage Levels

Graph	N	Damage	Latency	CARE	RL
RGG	50	2 nodes + 5 edges	0.2070	0.1311	0.3703
RGG	100	2 nodes + 5 edges	-0.1662	-0.1479	-0.0860
RGG	50	5 nodes + 10 edges	0.3301	0.2311	0.4241
RGG	100	5 nodes + 10 edges	-0.0412	-0.2010	-0.1274
ER	50	2 nodes + 5 edges	-0.0599	0.0000	-0.0091
ER	100	2 nodes + 5 edges	-0.0999	-0.0612	-0.0637
ER	50	5 nodes + 10 edges	-0.2106	-0.0717	-0.1020
ER	100	5 nodes + 10 edges	-0.0841	0.0000	0.0000

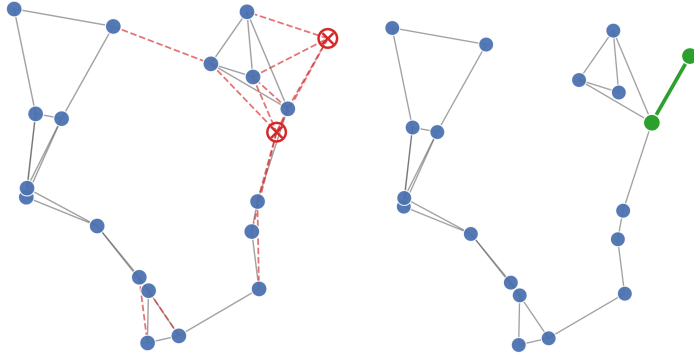


Fig. 2. CARE repair result on an ER graph with $N = 20$. From left to right: the damaged graph after removing 2 nodes and 5 edges, and the repaired graph.

4.5 Ablation Study of TI and GE

Two specialized testing scenarios isolate semantic understanding from structural logic. The experiments in Table 4 and Table 5 are evaluated on different damaged graph instances and should therefore be interpreted independently.

Enforcing Hard Constraints: Results in Table 4 show that while all configurations maintain connectivity, only those with the Graph Expert achieve minimal edge addition. Without GE, the system tends to add redundant edges because it lacks strict structural pruning.

Table 4. Ablation on Structural Constraints ($N = 50$, Instance A)

Method	Connected	ASPL ↓	Edges Added ↓	Correctness ↑
Full CARE	Yes	7.00	1	100%
w/o TI	Yes	7.00	1	100%
w/o GE	Yes	3.33	3	100%
Single LLM	Yes	3.33	3	100%

Semantic Grounding: In Table 5, disabling the Terminology Interpreter leads to a 0% strategy-correctness rate, because the model fails to map domain-specific terms to the intended strategy logic.

Table 5. Ablation on Semantic Grounding ($N = 50$, Instance B)

Method	Score $\uparrow$	ASPL $\downarrow$	Edges Added $\downarrow$	Correctness $\uparrow$
Full CARE	0.3447	7.76	1	100%
w/o TI	-0.1662	7.76	1	0%
w/o GE	0.3301	5.00	5	100%
Single LLM	-0.1274	5.33	6	0%

5 Conclusion

This paper presents CARE, a collaborative multi-agent framework for intent-aware graph connectivity restoration. The ablation study confirms that the Terminology Interpreter and Graph Expert play distinct and non-trivial roles. The current strategy space is intentionally small, offering transparent repair selection rather than attempting to replace all algorithmic methods. Compared with RL, CARE is advantageous when retraining is costly or distributions shift, whereas RL excels on stable graph families; the two approaches should therefore be viewed as complementary. Experiments on synthetic and real-world topologies demonstrate strong prompt-to-strategy alignment and robust performance without task-specific retraining. Future work will expand the strategy library, report broader multi-seed stability, and evaluate performance on larger networks.

References

- Cheng, Y., Chen, L., Yuan, Y., Wang, G., Li, B., Jin, F.: Strict and flexible rule-based graph repairing. *IEEE Transactions on Knowledge and Data Engineering* 34(7), 3521–3535 (2022)
- Dey, A., Ornik, M.: Post-disaster repair crew assignment optimization using minimum latency. In: *IEEE International Smart Cities Conference (ISC2)*. pp. 1–7 (2022)
- Lauer, A., Kosiol, J., Taentzer, G.: Empowering model repair: A rule-based approach to graph repair without side effects. In: *ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. pp. 831–840 (2023)
- Wang, Y., Pan, S., Li, C., Yin, M.: A local search algorithm with reinforcement learning based repair procedure for minimum weight independent dominating set. *Information Sciences* 512, 533–548 (2020)
- Xiao, Z., Zhang, D., Wu, Y., Xu, L., Wang, Y., Han, X., Fu, X., Zhong, T., Zeng, J., Song, M., Chen, G.: Chain-of-experts: When llms meet complex operations research problems. In: *International Conference on Learning Representations (ICLR)* (2024)
- Yu, X., Li, H., Wang, Y., Li, S., Shi, R., Ai, G., Pu, Z., Wu, W.: Clga: A collaborative llm framework for dynamic goal assignment in multi-robot systems. In: *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. pp. 2788–2794 (2025)
- Zhao, S., Chen, Z.: Optimizing graph navigability in dynamic environments: Local connectivity repair and incremental sampling. In: *International Conference on Frontier Technologies of Information and Computer (ICFTIC)*. pp. 1062–1065 (2025)